

Diplomado virtual PROGRAMACIÓN EN JAVASCRIPT

Guía didáctica – Módulo 1



```
der'],  
];  
otes);  
ods'] = $quotes;  
address;  
,  
ping_method']) && !  
ping_method']) &&  
ping_method'])['code']  
ta['lpa']['  
or_shipping_methods'];  
ication/json');  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
    type.pause = function (e) {  
    if (this.paused = true) {  
    if (this.$element.find('.next, .prev').length &&  
    this.$element.trigger($.support.transition.end  
    this.cycle(true)  
    }  
    this.interval = clearInterval(this.interval)  
    return this  
    }  
    Carousel.prototype.next = function () {  
    if (this.sliding) return  
    return this.slide('next')  
    }  
    Carousel.prototype.prev = function () {  
    if (this.sliding) return  
    return this.slide('prev')  
    }  
    Carousel.prototype.slide = function (type, next) {  
    var $active = this.$element.find('.item.active')  
    var $next = next || this.getItemForDirection  
    var isCycling = this.interval  
    var direction = type == 'next' ? 'left' : 'right'  
    var fallback = type == 'next' ? 'first' : 'last'  
    var that = this  
    if (!$next.length) {  
    if (!this.options.wrap) return  
    $next = this.$element.find('.item')[fallback]  
    }  
    if ($next.hasClass('active')) return (this.slidi  
    var relatedTarget = $next[0]  
    var slideEvent = $.Event('slide.bs.carousel', {  
    relatedTarget: relatedTarget,  
    direction: direction  
    })  
    this.$element.trigger(slideEvent)
```

Imagen: Pixabay



Imagen: Freepik

JS

Guía didáctica Fundamentos básicos



Competencia específica

Se espera que, con los temas abordados en la guía didáctica del módulo 1: Fundamentos básicos, el estudiante logre la siguiente competencia específica:

Conocer los conceptos básicos y aplicaciones del lenguaje de programación JavaScript en cuanto a variables, tipos de datos y operadores.



Contenidos temáticos

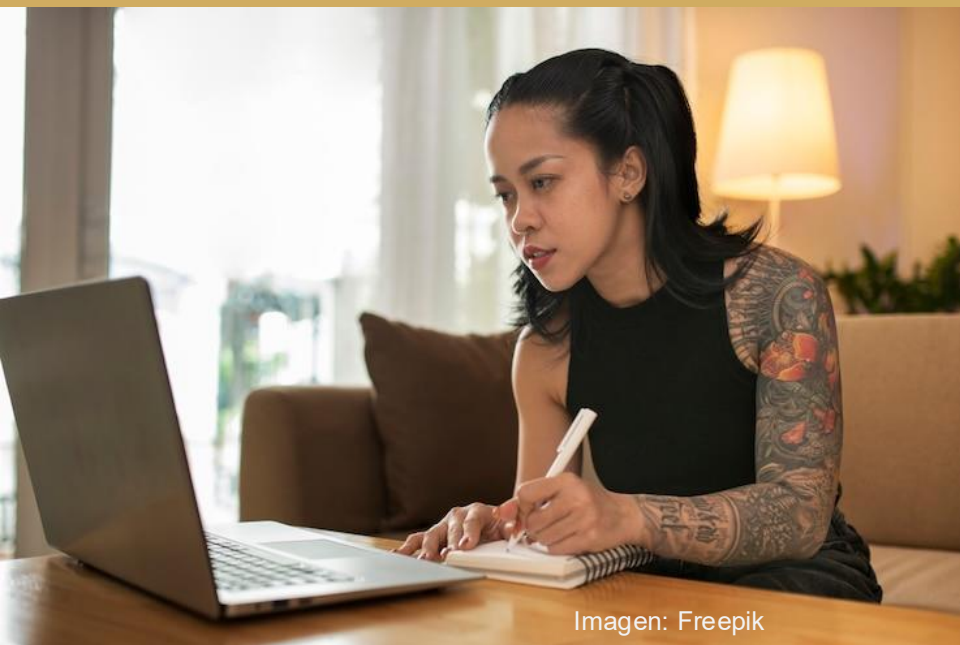


Imagen: Freepik

Los siguientes son los contenidos temáticos para desarrollar en la guía didáctica del módulo 1: Fundamentos básicos:

1

JavaScript

2

Variables

3

Tipos de datos

4

Operadores

Activación de saberes previos

Antes de iniciar con el estudio de la guía didáctica, reflexiona sobre las siguientes preguntas:

- ¿Cuáles son las características del lenguaje de programación JavaScript?
- ¿Qué estándar se utiliza para declarar variables con JavaScript?
- ¿Qué es una variable y cómo se declara en JavaScript?
- ¿Qué tipo de variables, datos y operadores se utilizan en JavaScript?

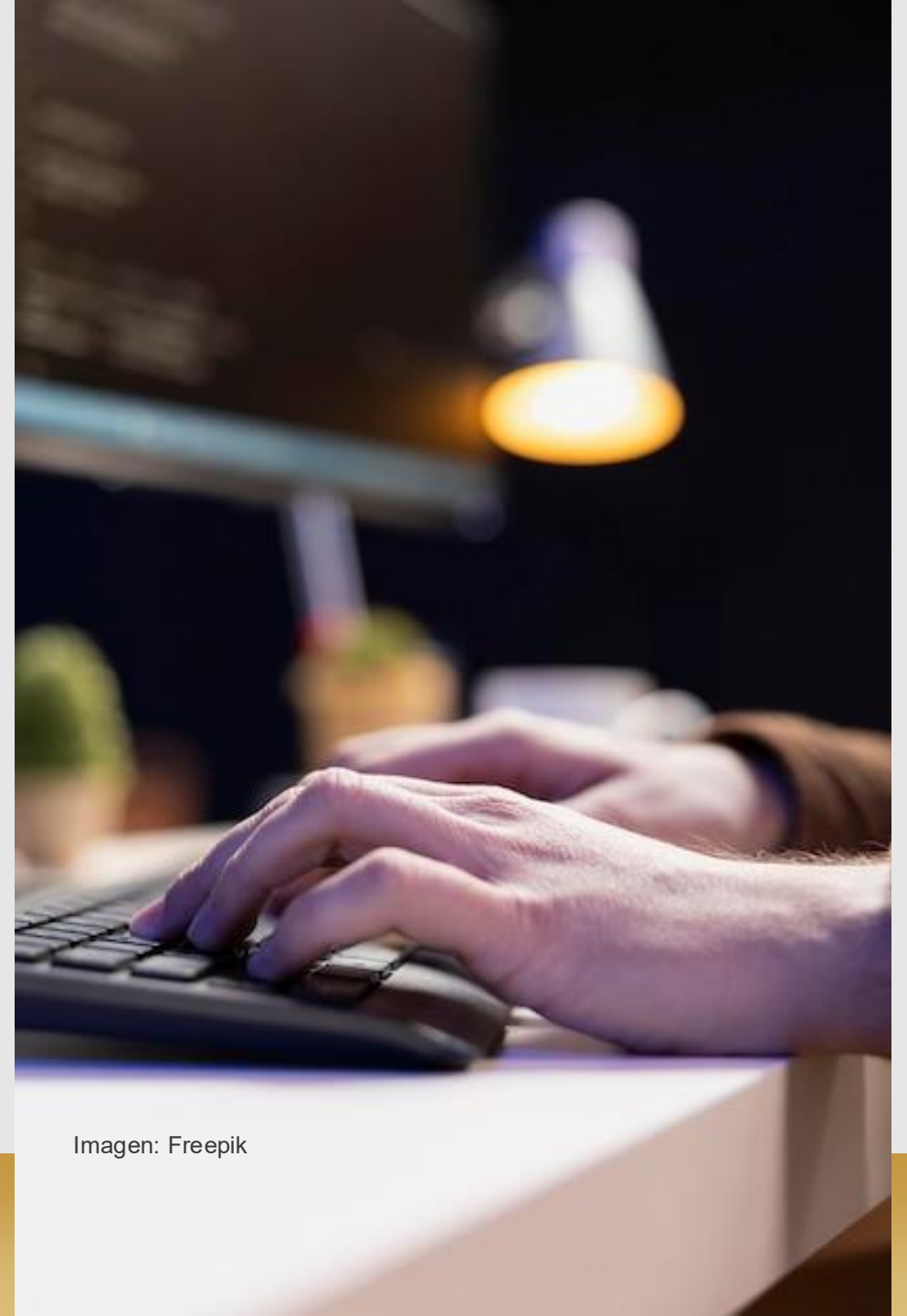


Imagen: Freepik

Tema 1:

JavaScript

«Sin requerimientos o diseño programar solo es el arte de agregar errores a un archivo vacío». Louis Srygley

JavaScript

JavaScript es un lenguaje de programación **orientado al cliente**, interpretado y orientado a objetos que se utiliza principalmente para crear **aplicaciones web interactivas y dinámicas**.



Imagen: Freepik

Orientado al cliente

- El navegador es el encargado de realizar el procesamiento del código.

Interpretado

- El navegador se encarga de leer cada línea de código e interpretarla sobre la marcha de ejecución.

Dinámico

- HTML por definición no puede generar interpretación en las páginas web, ahí es donde JavaScript permite generar dinamismo.

JavaScript

JavaScript es uno de los lenguajes de programación más usados y populares del mundo. **Nació en 1995** para darles interactividad a las páginas web, y desde entonces ha evolucionado hasta convertirse en un lenguaje de programación de propósito general. Dicho de otra forma: **se puede usar casi para cualquier cosa.**

¿Qué es programar?

Es el acto de construir un programa o conjunto de instrucciones para decirle a una computadora qué y cómo se debe hacer algo. No es diferente a cuando se «programa» el microondas, solo que, en vez de pulsar un botón, se generará texto. A este texto se le conoce como «código».

Características

JavaScript surge de la necesidad de generar dinamismo en las páginas web y que a su vez los usuarios y las empresas pudieran interactuar unos con otros.

Este lenguaje de programación tiene las siguientes características, que veremos con mayor detalle en las próximas páginas:

Lenguaje interpretado

Orientado a objetos

Lenguaje de alto nivel

Multiplataforma

Interacción de HTML y CSS

Gestión de eventos

Bibliotecas y *frameworks*

Lenguaje interpretado

JavaScript es un lenguaje de programación interpretado, esto significa que no necesita ser compilado antes de ejecutar el código.



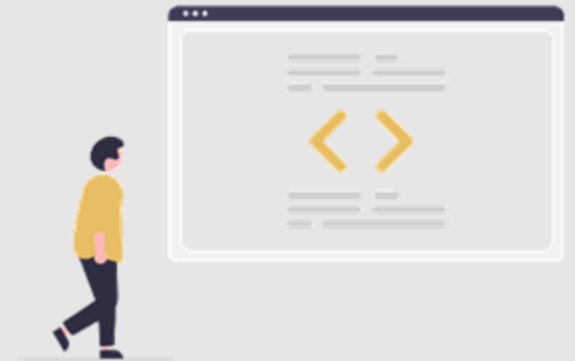
Orientado a objetos

JavaScript es un lenguaje de programación orientado a objetos, esto implica que utiliza objetos para representar datos y métodos.



Lenguaje de alto nivel

JavaScript es un lenguaje de programación de alto nivel, lo que significa que está diseñado para ser fácil de leer y escribir para los programadores humanos.





Multiplataforma

JavaScript puede ejecutarse en varios sistemas operativos y navegadores web diferentes, lo que lo hace muy versátil y fácil de implementar en una amplia variedad de proyectos, como aplicaciones móviles, de escritorio y aplicaciones web:

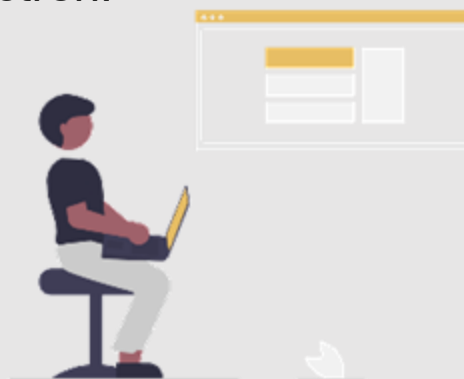
Aplicaciones móviles

Híbridas por medio de React Native.



Aplicaciones de escritorio

Multisistema por medio de Electrón.



Aplicaciones web

Frontend y backend.



Interacción de HTML y CSS

JavaScript se integra bien con HTML y CSS para crear páginas web dinámicas y atractivas.



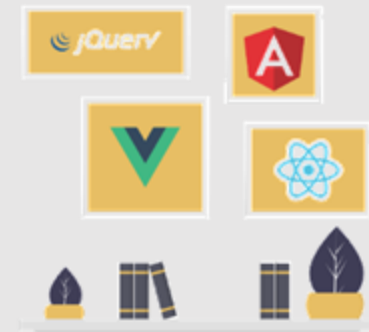
Gestión de eventos

JavaScript permite la gestión de eventos del usuario, como hacer clic en un botón o mover el cursor del *mouse*, lo que permite crear interacciones y experiencias de usuario más ricas.

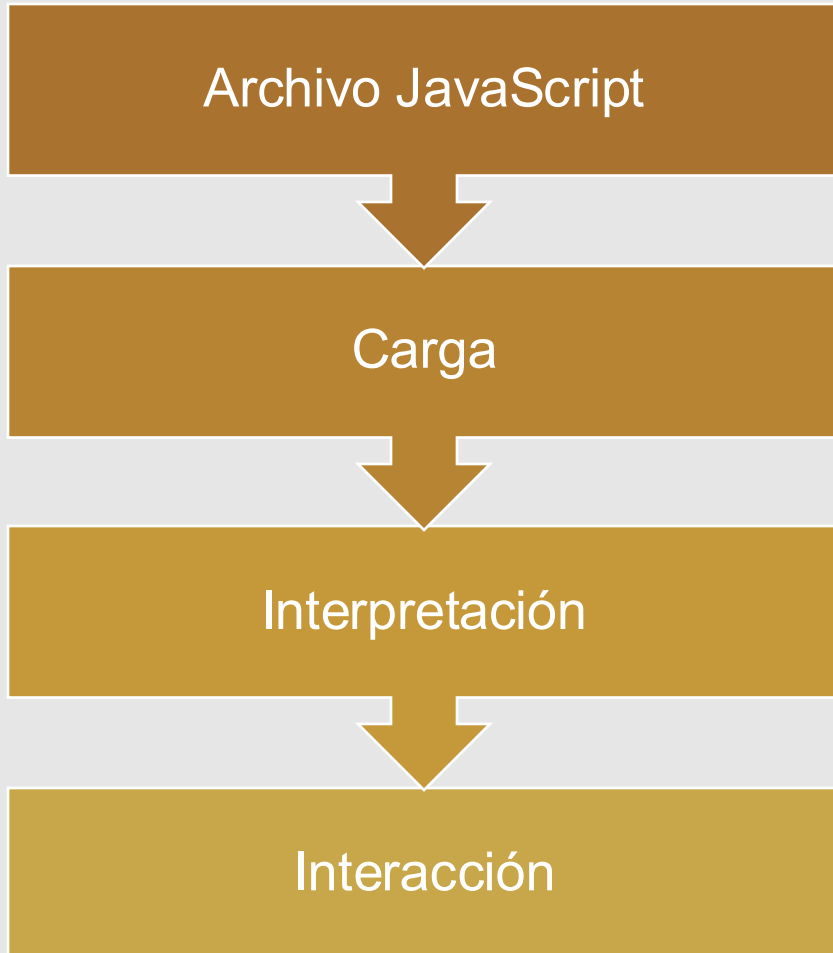


Bibliotecas y *frameworks*

JavaScript tiene una gran cantidad de bibliotecas y *frameworks* disponibles que facilitan el desarrollo de aplicaciones web y permiten a los desarrolladores hacer cosas más complejas con menos esfuerzo.



¿Cómo funciona?



El código JavaScript se incluye en un archivo HTML o se almacena en un archivo separado con extensión ".js".

Cuando un usuario visita una página web que incluye código JavaScript, el navegador carga el código JavaScript.

Una vez cargado, el navegador web interpreta y ejecuta el código JavaScript. El código puede interactuar con la página web, modificar elementos HTML, manejar eventos del usuario, validar formularios, enviar y recibir datos de un servidor, entre otras cosas.

El código JavaScript se ejecuta en el lado del cliente, lo que significa que se ejecuta en el navegador web del usuario. Esto permite una experiencia de usuario más interactiva, dinámica y rápida.

ECMAScript

Es un estándar de programación que define cómo los lenguajes de programación deben trabajar en los navegadores web y otros entornos de programación.

ECMAScript fue creado por la European Computer Manufacturers Association (**ECMA**) para estandarizar el lenguaje de programación JavaScript.

Se encuentran las siguientes versiones:



ECMAScript 1 -1997

ECMAScript 2 -1998

ECMAScript 3 - 1999

ECMAScript 4 - abandonada

ECMAScript 5 - 2009

ECMAScript 5.1 – 2011

ECMAScript 6 - 2015

ECMAScript 7 - 2016

ECMAScript 8 - 2017

ECMAScript 9 - 2018

ECMAScript 10 -
2019

ECMAScript 11 –
2020

ECMAScript 12 -
2021

Tema 2:

Variables

«La ciencia de hoy es la tecnología del mañana». Edward Teller

Variables

En **JavaScript**, una variable es un contenedor / espacio de memoria que se utiliza para almacenar un determinado dato. Estos datos pueden ser números, cadenas de texto, objetos, funciones y otros tipos de información.

```
1 institucion = "Politécnico de Colombia"
2 const nombre = "Diego Palacio"
3 const edad = 25
4 const pais = "Colombia"
5
6 let mensaje = "Hola"
7 var saludo = "Mundo"
8
9 const Persona = {
10     nombre: "Diego",
11     edad: 25,
12     pais: "Colombia"
13 }
14
15
```

El nombre de las variables debe ser un nombre descriptivo que se identifique y tenga contexto dentro del código.

Los nombres de las variables no pueden iniciar con números.

No pueden contener caracteres especiales o símbolos.

Existen 3 tipos de escritura de código y variables:

- a. **lowerCase** *recomendado
- b. **UpperCase**
- c. **snake_case**

Como en todo lenguaje, se deben tener en cuenta las palabras reservadas del lenguaje.

Tipos de variables

En JavaScript, hay tres formas de declarar variables: **var**, **let** y **const**. Aunque todas permiten declarar una variable:

var

let

const

```
1  institucion = "Politécnico de Colombia"
2  const nombre = "Diego Palacio"
3  const edad = 25
4  const pais = "Colombia"
5
6  let mensaje = "Hola"
7  var saludo = "Mundo"
8
9  const Persona = {
10     nombre: "Diego",
11     edad: 25,
12     pais: "Colombia"
13 }
14
15
```

Tipo var

Era la forma en que se declaraban las variables hasta **ECMAScript 5**. Casi ya no se usa porque es de forma global y tiene las siguientes características:

- Se puede redeclarar la variable.
- Se puede reescribir el valor de la variable.
- Tiene un alcance global, se puede hacer uso de la misma desde cualquier bloque de código. “{}”

Const y **let** son la forma moderna en que se declaran variables a partir de **ECMAScript 6 - 2015**.



```
1  var nombre = "Diego"
2  var apellido = "Palacio"
3  var edad = 25
4
5  var nombre = "Diego Palacio"
6
7  edad = 25
```

Tipo let

Posibilita declarar variables cuyo valor puede ser modificado. A diferencia de **var**, **let** nos permite ser más estrictos en la escritura de código, implementando la prohibición de la redeclaración de variables y limitando el alcance.

- No se puede redeclarar.
- Se puede reescribir el valor de la variable.
- Tiene un alcance de bloque.

```
1  let nombre = "Diego"
2  let apellido "Palacio"
3  let edad = 25
4
5  const salario = () => {
6      let horasTrabajadas = 40
7      let valorHora = 20
8      let salario = horasTrabajadas * valorHora
9      return salario
10 }
11
12 console.log(salario())
```


Tipo const

Permite declarar variables que nunca modificarán su valor. A diferencia de **var** y **let**, **const** es mucho más estricto en la escritura de código, implementando la prohibición de la redeclaración de variables y la reescritura de valor de las variables.

- No se puede redeclarar.
- No se puede reescribir el valor de la variable.
- Tiene un alcance de bloque.



```
1  const SALARIO_MINIMO = 1300000
2  const HORAS_LABORADAS = 160
3
4  const nombre = "Juan"
5  const apellido = "Perez"
6  const casado = false
```

Las variables tienen diferencias significativas en cuanto a su **alcance**, **mutabilidad** y **uso**.

A continuación, se explican las principales diferencias entre ellas:

Alcance

Mutabilidad

Redeclaración

```
1  institucion = "Politécnico de Colombia"
2  const nombre = "Diego Palacio"
3  const edad = 25
4  const pais = "Colombia"
5
6  let mensaje = "Hola"
7  var saludo = "Mundo"
8
9  const Persona = {
10     nombre: "Diego",
11     edad: 25,
12     pais: "Colombia"
13 }
14
15
```

Alcance

Las variables declaradas con **var** tienen un alcance de **función** o **global**, lo que significa que están disponibles en toda la función o en todo el documento, respectivamente.

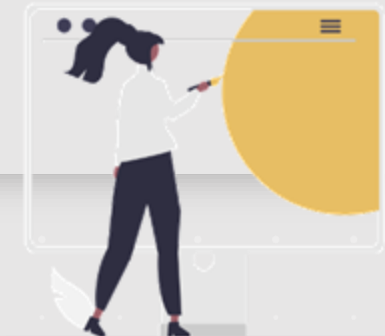
Las variables declaradas con **let** y **const** tienen un alcance de bloque, lo que significa que solo están disponibles dentro del bloque en el que se declaran, como un **if**, un **for** o una **función**.



Mutabilidad

Las variables declaradas con **var** y **let** son **mutables**, lo que significa que se pueden reasignar con un nuevo valor en cualquier momento.

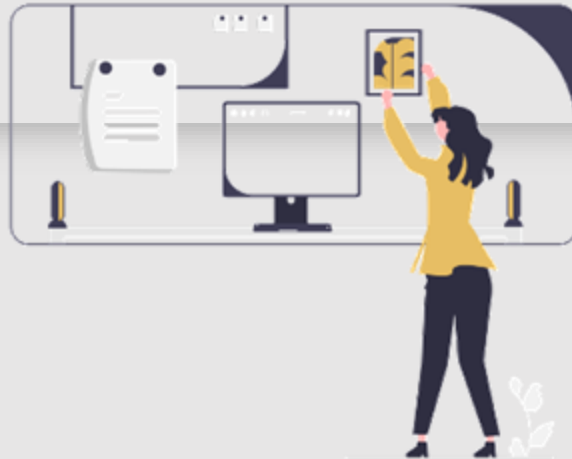
Las variables declaradas con **const** son **inmutables**, lo que significa que, una vez que se les ha asignado un valor, **no** se pueden **cambiar**.



Redeclaración

Las variables declaradas con **var** se pueden redeclarar.

Las variables declaradas con **let** y **const** no se pueden redeclarar.



En general, se recomienda utilizar **let** y **const** en lugar de **var** en **JavaScript** moderno, ya que **let** y **const** ofrecen un mejor control sobre el alcance y la mutabilidad de las variables.

Además, el uso de **const** siempre que sea posible promueve la inmutabilidad y puede ayudar a evitar errores y hacer que el código sea más fácil de entender.

Practica lo aprendido

Para poner en práctica lo visto acerca de las variables, realiza estos ejercicios:



Imagen: Freepik

Ejercicio 1

Crea 2 variables para almacenar el nombre y la edad de una persona.

Muestra el resultado de ambas variables por consola.

Ejercicio 2

Crea 2 variables constantes para almacenar los 7 primeros decimales de Euler y los 5 primeros decimales de pi.

Muestra ambos resultados por consola.

Practica lo aprendido



Ejercicio 3

Crea una variable que describa el error de volver a declarar una variable constante y de reemplazar su valor.

Explica.

Ejercicio 4

Crea 2 variables para intercambiar el valor de las mismas sin utilizar variables auxiliares, ¿cómo podrías hacerlo? Muestra los valores antes y después del intercambio.

Posteriormente, realiza el proceso con una variable auxiliar.

Tema 3:

Tipos de datos

«Primero resuelve el problema, después escribe el código». John Johnson

Tipos de datos

En **JavaScript**, al igual que en la mayoría de los lenguajes de programación, al declarar una variable y guardar su contenido, también le estamos asignando un tipo de dato, ya sea de forma **implícita** o **explícita**. El tipo de dato no es más que la naturaleza de su contenido: contenido numérico, contenido de texto, etc...



Imagen: Flaticon

Numbers

Strings

Booleanos

Arrays - objects

Undefined

Functions

Typeof

JavaScript a pesar de no de ser un lenguaje fuertemente tipado como **Java**, por ejemplo, cuenta con una tipificación dinámica, es decir, el tipado dependerá del valor y sus cambios en tiempos de ejecución.

Ya hemos creado variables y asignado diferentes tipos de datos, pero no hemos conocido ni validado efectivamente esos tipos de datos, aquí es donde entra **typeof**.

typeof se utiliza para determinar el tipo de datos de una variable o expresión. El operador devuelve una cadena de texto que indica el tipo de datos.

```
1  const PI = Math.PI
2  console.log(typeof PI) // number
3
4  const nombre = "Diego"
5  console.log(typeof nombre) // string
6
7  const casado = false
8  console.log(typeof casado) // boolean
9
10 const hijo = {
11     nombre: "Juan",
12     edad: 10
13 }
14 console.log(typeof hijo) // object
15
16 let fecha
17 console.log(typeof fecha) // undefined
```

Numbers

Almacenan valores numéricos, ya sean enteros o decimales, al igual que positivos y negativos.



```
1  const PI = Math.PI
2  const E = Math.E
3
4  const numeroUno = 23
5  const numeroDos = 23
6
7  const suma = numeroUno + numeroDos
8  const resta = numeroUno - numeroDos
9  const multiplicacion = numeroUno * numeroDos
10 const division = numeroUno / numeroDos
```



```
1  const PI = Math.PI
2  console.log('PI:', typeof PI) // number
3
4  const edad = 42.2
5  console.log('edad:', typeof edad) // number
```

Strings

Almacenan valores de texto o cadenas contenidos en comillas simples o dobles.



```
1  const nombreEstudiante = "Diego Palacio"
2  const ciudadEstudiante = "Bogotá"
3  const padreEstudiante = "Javier Palacio"
4  const madreEstudiante = "María Fernanda"
5
6  const ciudad = "Bogotá"
7  const departamento = "Cundinamarca"
8  const pais = "Colombia"
```



```
1  const nombreEstudiante = "Diego Palacio"
2  const ciudadEstudiante = "Bogotá"
3  const padreEstudiante = "Javier Palacio"
4  const madreEstudiante = "María Fernanda"
5
6  console.log(
7      'Nombre: ' + nombreEstudiante + '\n' +
8      'Ciudad: ' + ciudadEstudiante + '\n' +
9      'Padre: ' + padreEstudiante + '\n' +
10     'Madre: ' + madreEstudiante
11 )
```

Booleanos

Almacenan valores booleanos, es decir, verdadero o falso.



```
1  const casado = true
2
3  const tengoMascota = false
4
5  const tengoHijos = true
6
7  const tengoGatos = true
```



```
1  const casado = true
2
3  const tengoMascota = false
4
5  console.log("¿Estoy casado?", casado)
6  console.log("¿Tengo mascota?", tengoMascota)
7  console.log("Tipo de casado", typeof casado)
8  console.log("Tipo de tengoMascota", typeof tengoMascota)
```

Objects

Almacenan un conjunto de propiedades y valores relacionados o también conocidos como clave - valor. Los objetos se declaran utilizando la sintaxis de llaves.

```
1  const pais = {  
2      nombre: 'España',  
3      poblacion: 47000000,  
4      capital: 'Madrid',  
5      moneda: 'Euro',  
6      idioma: 'Español',  
7      region: 'Europa'  
8  }
```

```
1  const institucion = {  
2      nombre: "Politécnico de Colombia",  
3      ubicacion: "Medellín",  
4      fechaFundacion: "2013",  
5      departamento: "Antioquia",  
6      tecnicas: [  
7          "Asistente Administrativo",  
8          "Asistente en Desarrollo de Software",  
9      ]  
10 }  
11  
12 console.log(institucion)  
13 console.log(institucion.nombre)  
14 console.log(typeof institucion)
```

Arrays

Almacenan una lista de valores. Los arreglos pueden contener valores de cualquier tipo de datos, incluidos números, cadenas y booleanos. Para declarar un arreglo, se utiliza la sintaxis de corchetes.

```
1  const estudiantes = [  
2    "Diego",  
3    "Juan",  
4    "Pedro",  
5    "Maria",  
6  ]  
7  
8  const celulares = [  
9    "Samsung",  
10   "Iphone",  
11   "Huawei",  
12   "Xiaomi",  
13 ]  
14
```

```
1  const estudiantes = [  
2    "Diego",  
3    "Juan",  
4    "Pedro",  
5    "Maria",  
6  ]  
7  
8  const multiple = [  
9    [1, 2, 3],  
10   [4, 5, 6],  
11   [7, 8, 9],  
12 ]  
13  
14 console.log(estudiantes[0])  
15 console.log(multiple[0][0])  
16 console.log(typeof estudiantes)  
17 console.log(typeof multiple)
```

Undefined

Almacena la declaración de una variable no inicializada.



```
1 let cancion
2 let volumen
3 let duracion
4 var video
5
6 console.log(typeof cancion)
7 console.log(typeof volumen)
8 console.log(typeof duracion)
9 console.log(typeof video)
```



```
1 let cancion
2 let volumen
3 let duracion
4 var video
5 const cantante // Error
6
7 console.log(typeof cancion)
8 console.log(typeof volumen)
9 console.log(typeof duracion)
10 console.log(typeof video)
11
```


Funciones

Almacenan el bloque de ejecución de una función.



```
1  const nombre = "Diego"
2  const ciudad = "Madrid"
3
4  const mostrarDatos = () => {
5      console.log(`Mi nombre es ${nombre} y
6          vivo en ${ciudad}`)
7  }
```



```
1  const nombre = "Diego"
2  const ciudad = "Madrid"
3
4  const mostrarDatos = () => {
5      console.log(`Mi nombre es ${nombre} y
6          vivo en ${ciudad}`)
7  }
8
9  console.log(mostrarDatos)
10 mostrarDatos()
11 console.log(typeof mostrarDatos)
```

Practica lo aprendido

Para practicar lo visto sobre de los tipos de datos, realiza los siguientes ejercicios:



Imagen: Freepik

Ejercicio 1

Declara una variable de cada tipo de dato básico: número, cadena, booleano, null y undefined.

Muestra el tipo de dato de cada variable por consola.

Ejercicio 2

Declara un objeto que defina a una persona con nombre, edad y ciudad.

Muestra su tipo de dato por consola y la información contenida en el objeto.

Practica lo aprendido



Ejercicio 3

Crea una variable para que al utilizar typeof el resultado sea undefined.

Ejercicio 4

Crea una variable con el valor de NaN y verifica si su tipo de dato es number.

Tema 4:

Operadores

«Las personas que realmente toman en serio el *software* deberían hacer su propio *hardware*». Alan Kay

Antes de continuar con el tema 4,
reflexiona sobre las siguientes preguntas:

- ¿Qué son los operadores?
- ¿Qué operadores conoces?
- ¿Los operadores varían entre lenguajes de programación?



Imagen: Freepik

Operadores en JS

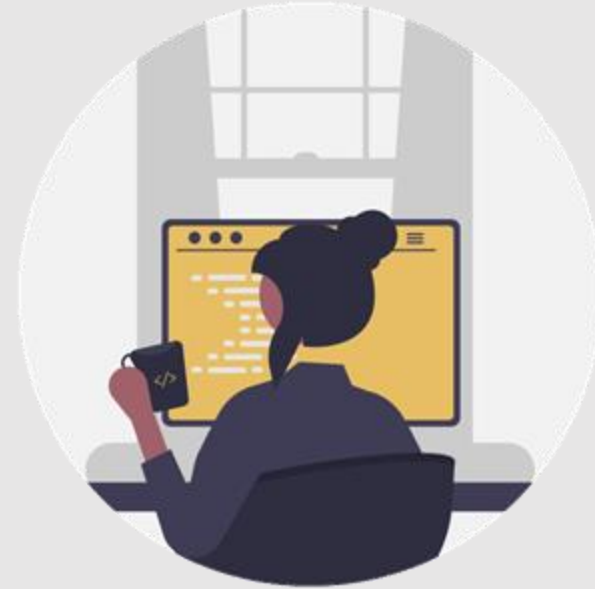
Operadores
aritméticos

Operadores de
incremento /
decremento

Operadores de
asignación

Operadores de
comparación

Operadores lógicos



Los operadores en **JavaScript** son **símbolos** o **palabras clave** que se utilizan para realizar operaciones en valores y variables.

Los operadores permiten **combinar**, **comparar**, **evaluar** y **modificar** valores, lo que es esencial en la programación.

Operadores aritméticos

Son utilizados para realizar operaciones matemáticas básicas, como **sumar**, **restar**, **multiplicar** y **dividir**. Algunos ejemplos son:

```
1 let numeroUno = 4
2 let numeroDos = 5
3
4 console.log("Suma: ", numeroUno + numeroDos)
5 console.log("Resta: ", numeroUno - numeroDos)
6 console.log("Multiplicación: ", numeroUno * numeroDos)
7 console.log("División: ", numeroUno / numeroDos)
8 console.log("Módulo: ", numeroUno % numeroDos)
9 console.log("Exponenciación: ", numeroUno ** numeroDos)
```

Debemos tener en cuenta la **precedencia** de **operadores**, esta regla determina el **orden** en que debemos realizar las operaciones matemáticas en una expresión.

Paréntesis: las expresiones dentro de los paréntesis se evalúan primero.

Exponentes: los exponentes se evalúan después de los paréntesis.

Multiplicación y división: estos operadores tienen la misma prioridad y se evalúan de izquierda a derecha.

Suma y resta: estos operadores también tienen la misma prioridad y se evalúan de izquierda a derecha.

Math

Es un **objeto** incorporado en JavaScript que proporciona una serie de propiedades y métodos matemáticos para realizar operaciones matemáticas en el código (Developer Mozilla, 2025).

Constantes

Métodos

Constante	Descripción
Math.E	Número Euler
Math.PI	Número pi
Math.SQRT2	Raíz cuadrada de 2
Math.LN2	Logaritmo natural de 2
Math.LN10	Logaritmo natural de 10

Método	Descripción
Math.max()	Número mayor
Math.min()	Número mínimo
Math.pow()	Potencia
Math.sqrt()	Raíz cuadrada
Math.random()	Número aleatorio
Math.round()	Redondeo cercano
Math.floor()	Redondeo inferior
Math.ceil()	Redondeo superior

Operadores de incremento

Es un operador utilizado para realizar operaciones matemáticas de **incremento** unario.

Operadores de decremento

Es un operador utilizado para realizar operaciones matemáticas de **decremento** unario.

Hay que tener en cuenta que existen **dos** maneras de utilizar el **incremento** y **decremento**: por medio de **prefijo** o **sufijo**. Cuando se utilizan como **prefijo**, el valor de la variable se actualiza **antes** de que se evalúe la expresión. Cuando se utilizan como **sufijo**, el valor de la variable se actualiza **después** de que se evalúe la expresión.

```
1 // Preflix
2
3 let numero = 12
4
5 console.log(numero++) // 12
6 console.log(numero) // 13
7
8 // Suffix
9
10 console.log(++numero) // 14
11 console.log(numero) // 14
```

```
1 // Preflix
2
3 let numero = 12
4
5 console.log(numero--) // 12
6 console.log(numero) // 11
7
8 // Suffix
9
10 console.log(--numero) // 10
11 console.log(numero) // 10
```

Operadores de asignación

Son utilizados para **asignar valores** a variables.

El operador de asignación básico es el signo **igual =**. Algunos ejemplos de operadores de asignación compuestos son:

```
1 let numero = 10
2
3 numero += 10 // Suma + Asignación = 20
4 numero -= 10 // Resta + Asignación = 10
5 numero *= 10 // Multiplicación + Asignación = 100
6 numero /= 10 // División + Asignación = 10
7 numero %= 10 // Módulo + Asignación = 0
```

= Asignación

+= Suma + Asignación

-= Resta + Asignación

*= Multiplicación + Asignación

/= División + Asignación

Operadores de comparación

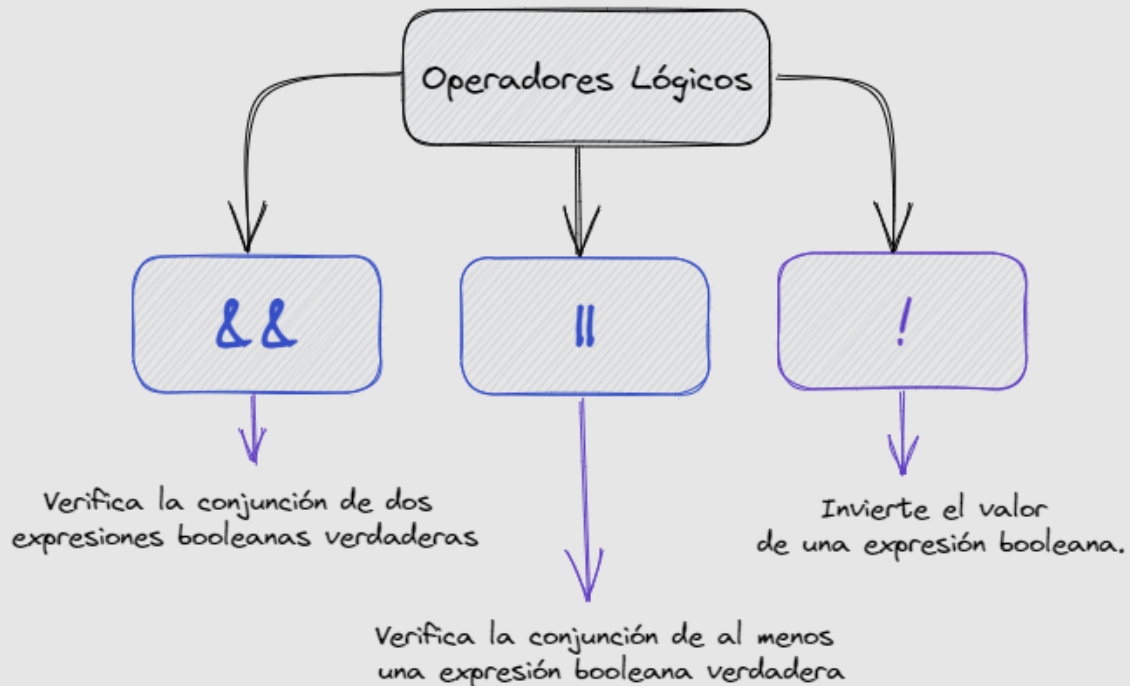
Son utilizados para comparar valores y devolver un valor **booleano true** o **false**. Algunos ejemplos son:

Operador	Descripción
==	Para comparar igualdad de valores
===	Para comparar igualdad de valores y tipos
!=	Para comparar desigualdad
!==	Para comparar desigualdad de valores y tipos
>	Para comparar si un valor es mayor
<	Para comparar si un valor es menor
>=	Para comparar si un valor es mayor o igual
<=	Para comparar si un valor es menor o igual

```
1 console.log(10 == "10") // true
2 console.log(10 === "10") // false
3
4 console.log(10 != "10") // false
5 console.log(10 !== "10") // true
6
7 console.log(10 > 11) // false
8 console.log(10 < 20) // true
9
10 console.log(10 >= 10) // true
11 console.log(10 <= 10) // true
12
```

Operadores lógicos

Se utilizan para **combinar dos** o más **expresiones booleanas** y devolver un resultado **booleano**.



&&

```
1 let estadoMatricula = true
2 let pagoMensualidad = true
3
4 let puedeEstudiar = estadoMatricula && pagoMensualidad
5
6 // console.log(puedeEstudiar) // true
```

```
1 let estaMatriculado = true
2 let pagoMensualidad = false
3
4 let puedeEstudiar = estaMatriculado && pagoMensualidad
5
6 // console.log(puedeEstudiar) // false
```

Operadores lógicos

||



```
1 let esMayor = true
2 let tieneCedula = false
3
4 let puedeVotar = esMayor || tieneCedula
5
6 // console.log(puedeVotar) // true
```



```
1 let esMayor = false
2 let tieneCedula = false
3
4 let puedeVotar = esMayor || tieneCedula
5
6 // console.log(puedeVotar) // false
```

&& !



```
1 let picoPlaca = true
2 let zonaPicoPlaca = false
3
4 let fotoMulta = picoPlaca && !zonaPicoPlaca
5
6 // console.log(fotoMulta) // true
```

|| !



```
1 let picoPlaca = false
2 let zonaPicoPlaca = true
3
4 let fotoMulta = picoPlaca || !zonaPicoPlaca
5
6 // console.log(fotoMulta) // false
```

Operadores de concatenación

La concatenación es el proceso de **unir** dos o más cadenas de texto en una sola **cadena**.

En JavaScript, la concatenación se realiza utilizando el operador de **suma** (+).

```
1 let nombre = "Diego"
2 let apellido = "Palacio"
3
4 const nombreCompleto = 'Hola ' + nombre + ' ' + apellido
5
6 console.log(nombreCompleto)
7
8 // Hola Diego Palacio
9
10 console.log('Hola ' + nombre + ' ' + apellido)
```

Template String

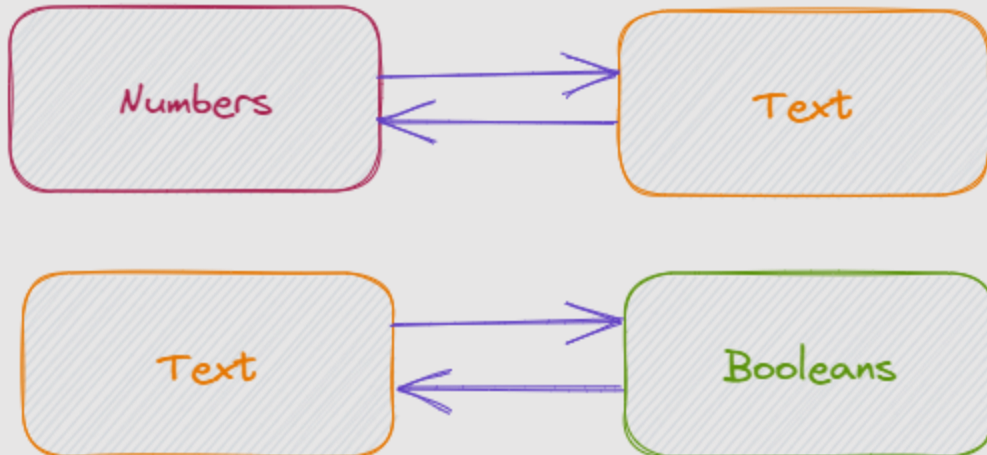
Es una forma de definir una **cadena** de texto que permite la **unión** de **variables** y **expresiones** dentro de ella de manera más sencilla y legible que con las comillas simples o dobles convencionales.

```
1 let nombre = "Diego"
2 let apellido = "Palacio"
3
4 const nombreCompleto = `Hola ${nombre} ${apellido} - Edad ${24}`
5
6 console.log(nombreCompleto)
7
8 // Hola Diego Palacio - Edad 24
9
10 console.log(`Hola ${nombre} ${apellido} - Edad ${24 + 1}`)
11
12 // Hola Diego Palacio - Edad 25
```

Conversión de variables

En JavaScript se pueden convertir diferentes tipos de variables utilizando diferentes métodos.

Conversión de Variables



Conversión de números a cadenas de texto: para convertir un número en una cadena de texto, se puede utilizar el método `toString()` o el operador de concatenación (+) junto con una cadena de texto vacía.

Conversión de cadenas de texto a números: para convertir una cadena de texto en un número, se puede utilizar el método `parseInt()` o `parseFloat()`.

Conversión de booleanos a cadenas de texto: para convertir un valor booleano en una cadena de texto, se puede utilizar el método `toString()` || `String()` o el operador de concatenación (+) junto con una cadena de texto vacía.

Conversión de cadenas de texto a booleanos: para convertir una cadena de texto en un valor booleano, se puede utilizar el operador de negación (!) dos veces, o el método `Boolean()`.

Ejemplo de código de conversión de variables



```
1  let numero = "213" // String("213")
2  console.log(numero) // "213"
3  console.log(typeof numero) // string
4
5  numero = parseInt(numero) // 213
6  console.log(numero) // 213
7  console.log(typeof numero) // number
8
9  let altura = "1.80" // String("1.80")
10 altura = parseFloat(altura) // 1.8
11 console.log(altura) // 1.8
12 console.log(typeof altura) // number
13
14 let esCasado = "true" // String("true")
15 esCasado = Boolean(esCasado) // true
16 console.log(esCasado) // true
17 console.log(typeof esCasado) // boolean
18
19 let tengoHambre = false // Boolean(false)
20 tengoHambre = String(tengoHambre) // "false"
21 console.log(tengoHambre) // "false"
22 console.log(typeof tengoHambre) // string
```




Practica lo aprendido

Pon en práctica lo visto acerca de operadores resolviendo los siguientes ejercicios:

Ejercicio 1

Declara 2 variables para almacenar la longitud y el ancho de un rectángulo.

Calcula el área y el perímetro, luego imprime ambos valores.

Ejercicio 2

Declara una variable para almacenar una temperatura en grados Celsius y conviértela a grados Fahrenheit.

La fórmula es

$$F = C * 9/5 + 32.$$

Ejercicio 3

Declara variables para el capital, la tasa de interés y el tiempo.

Calcula el interés simple usando la fórmula

$$\text{Interés} = \text{Capital} * \text{Tasa} * \text{Tiempo}$$

Ejercicio 4

Declara 3 variables para almacenar 3 números.

Calcula el promedio de estos números y luego imprímelo.

Practica lo aprendido



Ejercicio 5

Declara una variable para almacenar una cantidad de minutos y conviértela a horas y minutos restantes.

Imprime ambos valores.

Ejercicio 6

Declara 2 variables enteras y usa operadores relacionales para compararlas.

Imprime los resultados de cada comparación.

Ejercicio 7

Crea una operación aritmética donde se haga uso de todos los operadores matemáticos en repetidas ocasiones con los números que tú determines.

Ejemplo: $(9 / 2) + 8 * 2 / 1 - (2 + 2)$

Ejercicio 8

Calcula el precio final de un producto después de aplicar un descuento.

Referencia bibliográfica

Developer Mozilla. (2025). *Math*. Developer Mozilla.

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math

Esta guía fue elaborada para ser utilizada con fines didácticos como material de consulta de los participantes en el diplomado virtual en PROGRAMACIÓN EN JAVASCRIPT del Politécnico de Colombia, y solo podrá ser reproducida con esos fines. Por lo tanto, se agradece a los usuarios referirla en los escritos donde se utilice la información que aquí se presenta.

GUÍA DIDÁCTICA 1

M3-DV114-GU01

MÓDULO 1: FUNDAMENTOS BÁSICOS

© DERECHOS RESERVADOS - POLITÉCNICO DE
COLOMBIA, 2025
Medellín, Colombia

Proceso: Gestión Académica Virtual

Realización del texto: Diego Alejandro Palacio, docente

Revisión del texto: Comité de revisión

Diseño: Comunicaciones

Editado por el Politécnico de Colombia

